

sol4_fixer_deep_agents

Configuration port. It does **not** run the fixer.

This is the role graph. The live harness is `sol4_fixer_agent_sdk`.

No `fixer.py`, `gates.py`, `doers.py`, **or** `tests.task test` **is** `--table-only`.

What it proves

role	writes	scope	
orchestrator	no	nothing	
code_implementer	yes	app/**, src/**	denied tests/**
judge	no	nothing	

Live repair is `sol4_fixer_agent_sdk.backend(contract)` **exists**. `main` **never** calls it.

Learning objectives

- Read `LOOPS["fixer"]` (three roles, no planner)
- Hand the coder a scoped write tool
- Hand the judge `read_file` only
- Refuse `tests/**` with a sentence
- Name the folder that actually repairs `broken-pr`

Starting architecture

```
python loop.py --table-only
└─ three RolePlans

python loop.py --repo CRM
└─ create_deep_agent subagents
    code-implementer: read_file + write_code_implementer
    judge: read_file
```

No while loop. No junit. No comment.

Scoped write tool

```
@tool(f"write_{role.name}")
def write(path: str, content: str) -> str:
    try:
        scope.check(path)
    except ScopeViolation:
        return f"REFUSED. {role.name} may write {allowed}. {path} is outside that scope."
```

Deny tests/**. **Allow** app/**, src/**. Refusal is a string, not an exception.

Commands

```
cd solutions/sol4_fixer_deep_agents
python loop.py --table-only --repo ../../work/northwind-field-crm
python loop.py --repo ../../work/northwind-field-crm
task test
```

Missing repo plus `--table-only`: prints declared scopes, exit 0. Without `--table-only`, `ContractError` raises.

Expected table

Judge prints `no`. Code implementer prints `yes` with `deny tests/**`.

If judge prints `yes`, stop.

Troubleshooting

SYMPTOM	CAUSE	FIX
Expected a green branch	config port	Agent SDK folder
Judge writes yes	write tool handed to judge	tools = [read_file]
Raises on missing repo	forgot --table-only	table-only is the self-check

Validation

- [] table: three roles
- [] judge no
- [] coder denied tests/**
- [] you can name sol4_fixer_agent_sdk as the live loop

Spillwave Solutions | spillwave.com

Recap

Same table, Deep Agents enforcement knob. This is graph without a loop. Use the Agent SDK folder when you want a green branch or an honest comment.

Spillwave Solutions | spillwave.com

Prerequisites

```
cd solutions/sol4_fixer_deep_agents  
python loop.py --table-only
```

DEFAULT_LOOP in this copy is "implementer". So is sol1's. Both inherited it from the shared `roleplan.py` this repo deleted, and neither changed it. `loop.py` sets `LOOP = "fixer"` and passes it at every call site. Without that you get five roles instead of three.

The rule is that a caller names its loop. A test in each folder pins it, because the default is one edit away from being wrong everywhere.

Files in this folder

```
SPEC.md Taskfile.yml  
adapter.py contract.py loop.py  
roleplan.py roles.py write_scope.py
```

Missing on purpose: `fixer.py`, `doers.py`, `gates.py`, `tests/`.

Why three roles, not five

The work is already defined by what is red. No planner. No test implementer. The judge reads junit. The coder may write `app/**` and may not write `tests/**`.

Spillwave Solutions | spillwave.com

Final checklist

- `[] LOOP = "fixer"`
- `[] table: three rows, judge no`
- `[] coder write tool refuses tests/**`
- `[] live repair is sol4_fixer_agent_sdk`

Spillwave Solutions | spillwave.com